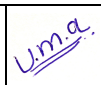
# Assessment Coversheet

| | | | |
|---|---|---|---|
| **Programme Title and Cohort Number** | **IP – B.Sc. in Computing C1** | **Name of Lecturer** | **Owen Sacco** |
| **Module Title and Module code** | **BSCG6_21.1_32_6 Advanced Programming** | **Number of Tasks in Assignment** | **2** |
| **Date of issue to students** | **01/09/2026** | **Submission Deadline (each task)** | **Task 1 – 30/09/2026<br>Task 2 – 30/10/2026** |
| **Verifier name** | **Joshua Bugeja** | **Verifier Signature** |  |
| **Date verification** | **1 Sep 2026** | | |

**NB.** *Assessment task brief to be included as separate document in format of lecture's choice (if unconventional format is required, i.e., not Word, PDF, PPT, Excel, etc., kindly first check with IDEA).*

**Sections A and/or B are to be filled in by Student/Students as appropriate.**

**KINDLY INCLUDE ALL DETAILS INDICATED BELOW AS APPROPRIATE AND SUBMIT WITH YOUR ASSIGNMENT. REMEMBER THAT THE ASSIGNMENT MUST BE SUBMITTED ELECTRONICALLY ON CANVAS PLATFORM IN THE RELEVANT SECTION.**

| **Section A: Individual Task** | | | | **Word count used** | |
|---|---|---|---|---|---|
| **Read and sign** | | | | | |
| I certify that the attached assignment is my own work. Information drawn from different sources has been appropriately referenced and fully acknowledged in the relevant section. Referencing throughout this assignment follows **British Harvard Referencing Style**. | | | | | |
| **Name of student** | **Mitesh Umretiya** | **Student's signature** |  | **Date of Submission** | **30/10/2026** |

| **Section B: Group Task** | **Word count used** | |
|---|---|---|
| **Read and sign** | | |
| We certify that the attached assignment is our own work. Information drawn from different sources has been appropriately referenced and fully acknowledged in the relevant section. Referencing throughout this assignment follows **British Harvard Referencing Style**.<br><br>*We confirm that all group members have contributed equally to complete this assignment. In case of disagreement with this statement group members are to advise the lecturer.* | | |

| **Name of students** | | **Students' signatures** | **Date of Submission** |
|---|---|---|---|
| | | | |
| | | | |
| | | | |
| | | | |
| | | | |

# Image Catalogue

Technical Report

BSCG6_21.1_32_6 Advanced Programming

# 15.1 System overview

Image Catalogue is an application that provides users with a simple method for organizing their own collections of photographs. When the user selects a directory and clicks "Add" (or "Scan"), the application will automatically start scanning all files within this directory in the background; once finished, it creates a thumbnail view of each image. In addition to the thumbnails, the user can see information such as the image's file size, format, and dimensions. All of these pieces are displayed in a sortable table where the user has the ability to select one piece at a time and get a larger preview of what they have chosen. Each piece may be given a Title, Tag(s), and Star Rating (on a scale from 0 to 5). The user may also search for specific images using either its Name, Title or Tags. Additionally, when the user wants to organize their images in some other way than how they were initially organized (e.g., after renaming them), they may close the Application and reopen it later by simply opening the .catalogue file which contains all of the previous information.

A short audio cue plays when the user has completed a Scan.

All of the above features allow the user to create a very large database of images (upwards of 10,000 images) and easily find individual images based upon criteria that include but is not limited to: Title, Tag, Name, Dimensions, Format, etc.

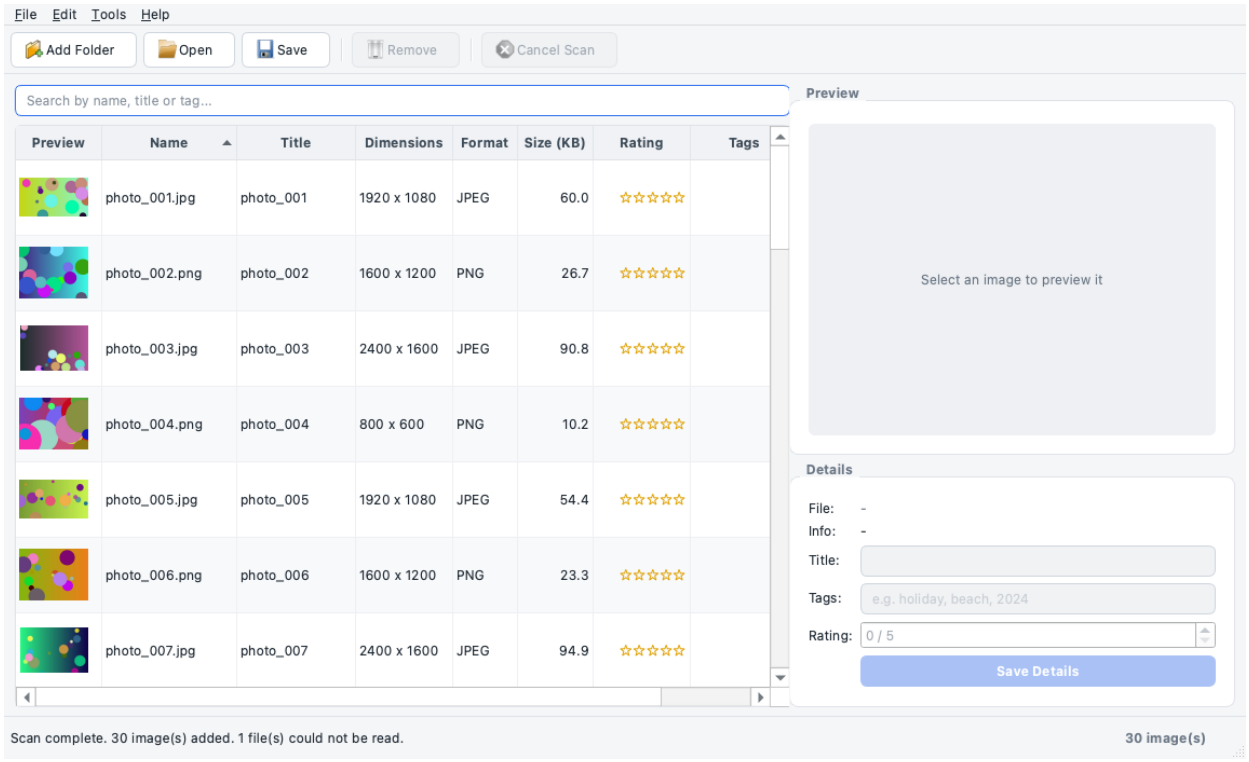



*Figure 1: Main window after scanning 30 images*

# 15.2 Architecture

The project uses MVC, adapted to Qt's model/view framework (see the diagram in Appendix A).

| Layer | Module / class | Responsibility |
|---|---|---|
| Model | model/image_item.py: ImageItem, validation functions | One image's data; rules for valid title, tags, rating |
| Model | model/catalogue.py: ImageCatalogue | The collection: add, remove, edit, search, save/load JSON |
| Model | model/processing.py | Find image files; read details and make thumbnails |
| View | views/main_window.py: MainWindow, PreviewLabel, ImageTable | Widgets, menus, toolbar, dialogs; emits custom signals |
| View | views/theme.py | The light theme: Fusion style, a fixed light palette and one style sheet for buttons, inputs, table and status bar |
| View (Qt adapter) | views/image_table_model.py: ImageTableModel, ImageFilterProxy | Connects the catalogue to QTableView; sorting and filtering |
| Controller | controller/main_controller.py: MainController | Reacts to view signals, updates the model, runs the worker |
| Worker | workers/scan_worker.py: ScanWorker, ScanStats | Background scanning and thread-safe progress counting |

# Important Decisions:

- **No Qt Imports:**

  The model does not import Qt modules. So, this model can be tested for its functionality without even opening one window. Also, the same process_image function will be utilized by both the application and the test framework.

- **Signals from View Only:**

  The Save Details button will only emit a signal. For instance, the Save Details button emits the signal details_submitted(title, tags, rating) and then the controller will ask the model if it validates and saves the title, tags and rating. Therefore, as stated before, there should be no business logic in any event handler of a GUI (Task 3).

- **Qt Table Model and ImageCatalogue?**

  QTableView requires a QAbstractTableModel to determine whether to add or remove rows. Rather than putting my data into a Qt class, I maintained ImageCatalogue as pure Python and implemented a thin adapter. If I had stored my data directly into a QStandardItemModel, I could have written much less code; however, I would have mixed my data with Qt classes and made testing more difficult.

- **Custom Signals Used:**

  MainWindow utilizes 9 custom signals (for example, open_folder_requested, search_changed(str), details_submitted(str, str, int)). ScanWorker utilizes 4 custom signals (for example, progress(int, int), item_ready(object), scan_finished(int, list, bool), scan_failed(str)).

- **Error Handling:**

  Invalid input raises a ValidationError and the controller displays an error message. File issues that occur during scanning cause a CatalogueError with an easily

understandable description. Broken images are recorded in the results database and skipped during scanning.

## 15.3 GUI design decisions

- **QMainWindow** gives the standard menu bar, toolbar and status bar layout that users already know.
- **Light theme with clear buttons (`views/theme.py`).** In the first version the toolbar actions showed as plain text, which didn't look like buttons. I switched to Qt's Fusion style with a fixed light palette, so the app looks the same on macOS and Windows and stays light even when the Mac is in dark mode. Every toolbar button now has an icon and text, a border and a hover highlight. The icons are Qt's built-in standard icons, so there are no extra image files to package.
- **Colour means something.** Blue is only used for the main action in the form (Save Details). Red is only used for actions that remove or stop something (Remove, Cancel Scan). Disabled buttons turn grey, so it's clear what you can do at any moment. For example, Remove stays grey until an image is selected.
- **Rating Stars** make ratings easy to read at a glance. The empty table shows "No images yet. Click Add Folder to start." instead of a blank area, or "No images match your search." when a search finds nothing.
- **All the styling is in one file.** The window code only gives widgets names like `primaryButton` or `dangerButton`, and `theme.py` decides what those look like. The look can change without touching the window's logic.
- **QTableView rather than a QListWidget of thumbnails:** the user wants to compare details (size, rating, tags) and sort by them, and a table does that well. The thumbnail is in the first column so it's still visual.
- **QSplitter** between the table and the preview so the user can choose how much space each gets.
- **QFormLayout** for the details, since label/field pairs are what forms are for.
- **QSpinBox for rating (0-5) and `setMaxLength(100)` on the title:** these stop invalid values before they happen. The model validates anyway, in case data comes from somewhere else such as a hand-edited file.
- **Progress bar and Cancel button in the status bar,** only visible while scanning, so they don't clutter the window the rest of the time. Add Folder and Open Catalogue are disabled during a scan so the user can't start two at once.
- **Confirmation dialogs** only for actions that lose something (remove, closing with unsaved changes). Information that doesn't need a decision goes in the status bar instead of a popup.
- **Keyboard shortcuts** (Ctrl+O, Ctrl+S, Delete, Esc) for faster use.
- **Sound (QtMultimedia, `QSoundEffect`):** a scan of a large folder can take a while, so the user may switch to another window. The chime tells them it's finished without having to keep checking. It can be turned off in the Tools menu. I generated the sound myself (`tools/make_sound.py`), so there are no copyright issues.

## 15.4 Concurrency design

The issue. Creating thumbnails involves decoding large photos. As described above, in the benchmark the process of creating thumbnails for 40 photos took approximately 2.1 seconds as they were processed sequentially. Therefore, processing all photos in a large

number of photos could take many seconds. When I created my initial version of the program (commit "Add main window, table model and controller (scan runs on GUI thread)"), I placed the code that decoded the images into a single loop within the controller. This resulted in no ability to have the window redisplay or click buttons while it was running. It also caused the application to show the spinning wheel on macOS. For these reasons, the threaded processing of scans was added on the feature/threaded-processing branch.

**Technique.**

- A ScanWorker (a QObject) was moved onto a QThread created by the controller with moveToThread. When the thread started, the ScanWorker's run() slot would start.
- In run(), a ThreadPoolExecutor using four threads would then concurrently decode multiple images.
- After every image has finished, the worker would emit an item_ready(item) signal and a progress(done, total) signal.
- Finally, it would emit a scan_finished(...) signal and exit. At this point, a short sound would play; and if there had been any skipped files, they would be reported.

How do we get data back to the user interface? Qt widgets may only be interacted with from the GUI thread. Because both the controller and worker live in different threads than the controller, Qt will automatically use queued connections. In other words, the parameters passed to a signal are sent to the event queue of the GUI thread where they can be picked up and used in a slot. The worker does not touch either a widget or the catalogue.

**Synchronisation and thread safety (Task 7).**

| Shared resource | Who uses it | What could go wrong | Protection |
|---|---|---|---|
| `ScanStats` counters (processed count, failure list) | All 4 pool threads write; the worker thread reads | `processed += 1` is really read, add, write. Two threads can read the same value and both write back value+1, so one update is lost. The progress bar would never reach 100%. | `threading.Lock` around every read and write (`with self._lock:`) |
| The catalogue and the table model | Only the GUI thread | If a worker thread added rows while the table was repainting, Qt could crash or show wrong rows | **Controlled ownership:** only the GUI thread touches them; workers send items by signal |
| Cancel flag | GUI thread sets it; pool threads read it | A plain variable could be read in an inconsistent state | `threading.Event`, which is designed to be shared between threads |

Evidence: `tools/race_demo.py` runs 4 threads x 10,000 increments. **Without a lock it gave 10,006 instead of 40,000. With the lock it gave exactly 40,000.** The automated test `test_stats_counter_is_correct_with_many_threads` checks the real `ScanStats` with 8 threads.

**Cancelling.** `cancel()` is called directly from the GUI thread, not through a signal. A queued signal to the worker would sit in the worker thread's queue until `run()` finished, which is too late. The event is safe to set from any thread. When it is set, images not yet started

are cancelled and those in progress are allowed to finish, so no half-written thumbnails are left behind.

**Responsiveness evidence.** During manual test MT04, a 20 ms timer on the GUI thread kept firing for the whole 200-image scan (see `docs/evidence/gui_walkthrough_results.txt`). If the GUI thread had been blocked, the timer couldn't have fired.

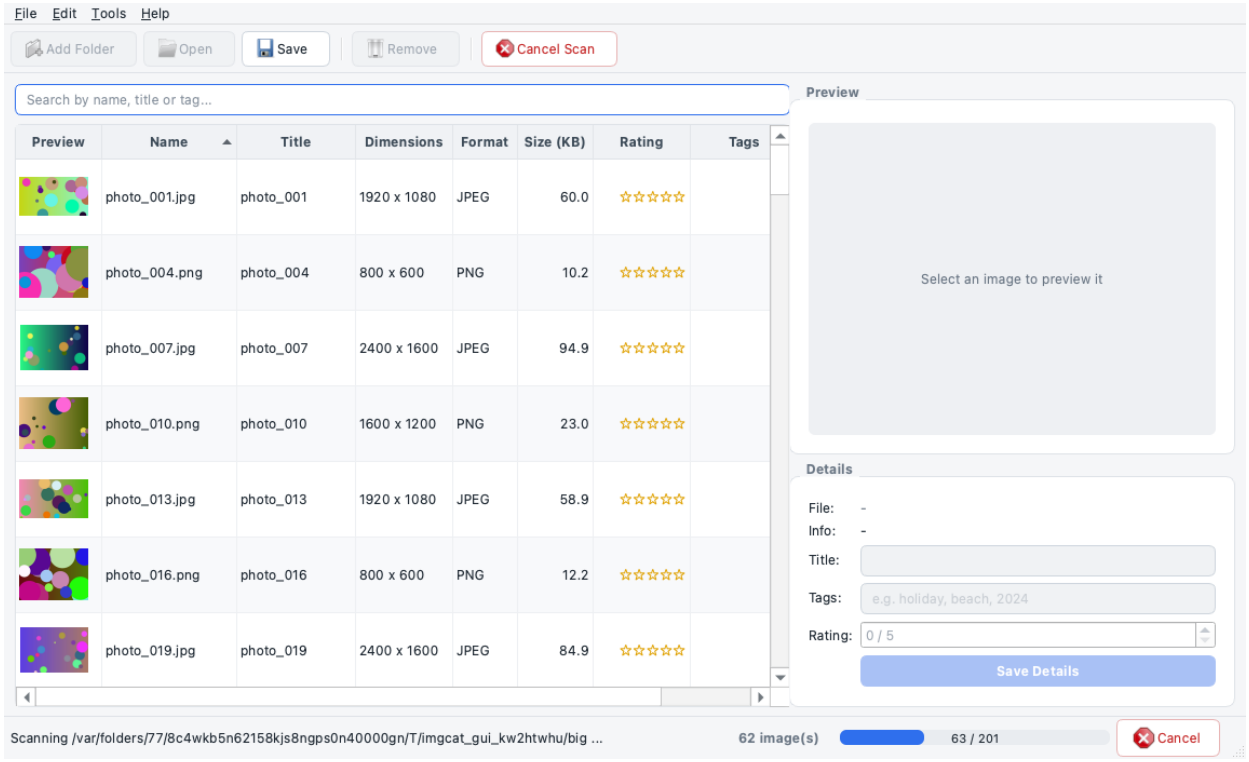



*Figure 2: Progress bar and Cancel button during a scan*

**Performance.** See Appendix C. On my 8-core Mac, 4 threads gave a 3.25x speed-up and 8 threads gave 4.5x. That was faster than processes (2.69x with 4), because Pillow releases the GIL and threads avoid the cost of pickling. I chose threads because they were faster here and are simpler with Qt signals and packaging.

## 15.5 Testing

See Appendix B: 27 manual tests, 68 automated tests (all passing), and a debugging log of 5 defects found and fixed.

- **Automated:** `python -m pytest -v` gives 68 passed in under a second. The tests cover validation (normal, boundary and invalid values), the catalogue, file handling, thumbnails, the worker and the locked counter (8 threads x 5,000 updates).
- **Manual:** the 27 GUI tests were run on the real window with `tools/gui_walkthrough.py`, which saves a screenshot for each important step to `docs/evidence/`. Sound, keyboard shortcuts and the packaged app were checked by hand.
- **Defects:** the most interesting one was D5. Rescanning a folder reported "30 image(s) added" when nothing was new. The worker can't know which images are duplicates, because that check happens on the GUI thread. The fix was to count additions in the controller. It is a separate "Fix:" commit in the Git history.

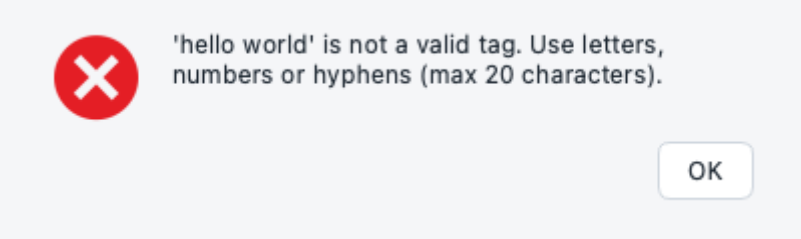



*Figure 3: Validation error for an invalid tag*

## 15.6 Source-control workflow

The project was built up in stages, with a commit at each step:

1. **main**: project setup and design spec, then the model (ImageItem, validation, ImageCatalogue) with tests, then image processing with tests, then the sound and sample-image tools.
2. **main**: the first GUI, with the scan running on the GUI thread (it froze the window).
3. **feature/threaded-processing** branch: added `ScanWorker` with the thread pool and the locked counter, then switched the controller to a background `QThread` with progress and cancel. Merged back into main with `--no-ff`, so the branch is visible in the history.
4. **feature/window-title** branch: a **controlled merge conflict**. Both main and the branch changed the same `setWindowTitle(...)` line. `git merge` stopped with `CONFLICT (content): Merge conflict in catalogue/views/main_window.py` and added `<<<<<<< HEAD`, `=======` and `>>>>>>>` markers. `HEAD` is main's version and the part after `=======` is the branch's. I resolved it by keeping the branch's title ("Image Catalogue - Photo Organiser"), deleted the markers, ran the tests (68 passed) and committed the merge. The full output is in Appendix D.
5. **feature/testing** branch: the benchmark, the GUI walkthrough script, the D5 bug fix as its own `Fix:` commit, the PyInstaller configuration (with the macOS `.app` bundle), and the test, performance and report documents.
6. **Pull request and merge:** `feature/testing` was merged into main on GitHub through pull request #1, then the GitHub screenshots were added on main.
7. **feature/ui-polish** branch: the visual polish (light theme, button icons and colours, gold stars, empty-table message), with the tests and the GUI walkthrough re-run so the screenshots show the new look. Merged into main through pull request #2.

**Remote:** all four branches were pushed to a GitHub repository (`miteshumretiya/image-catalogue`) with `git push -u origin ...`. Pull request #1, "Add testing, benchmark, packaging and documentation", contained 10 commits and 25 changed files, with a description of what it adds and how it was tested. After checking the "Files changed" tab I merged it on GitHub, then ran `git pull` to bring the merge commit back to my Mac. I kept the feature branches on GitHub instead of deleting them, so the history stays visible.

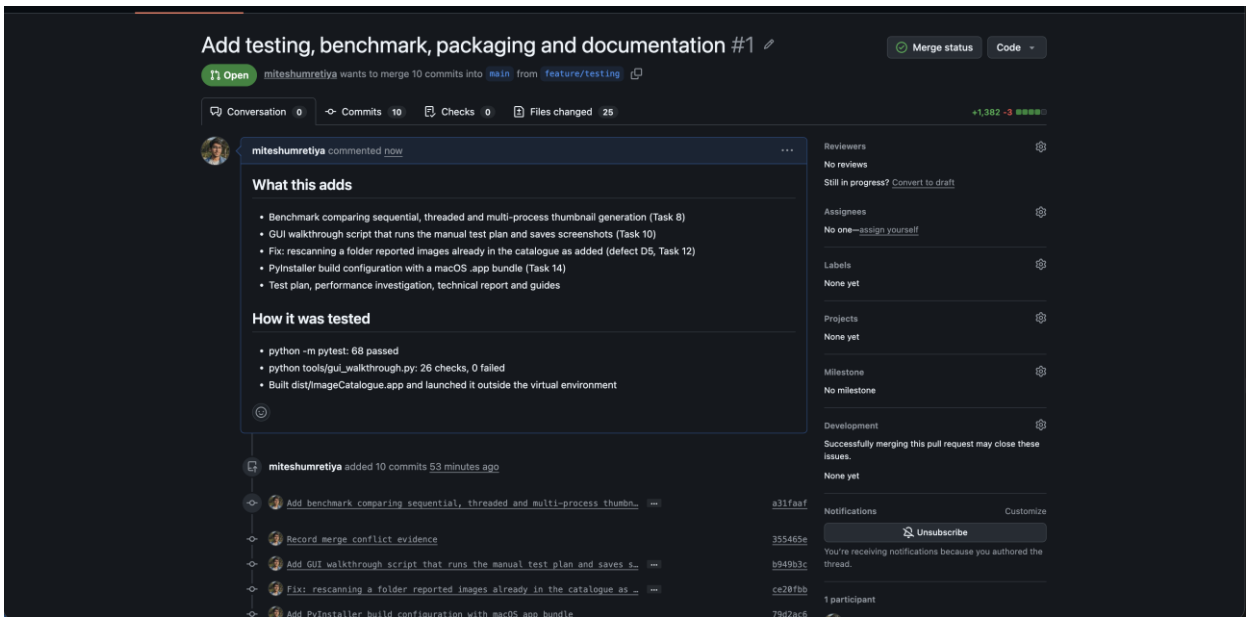



*Figure 4: Pull request #1 on GitHub*

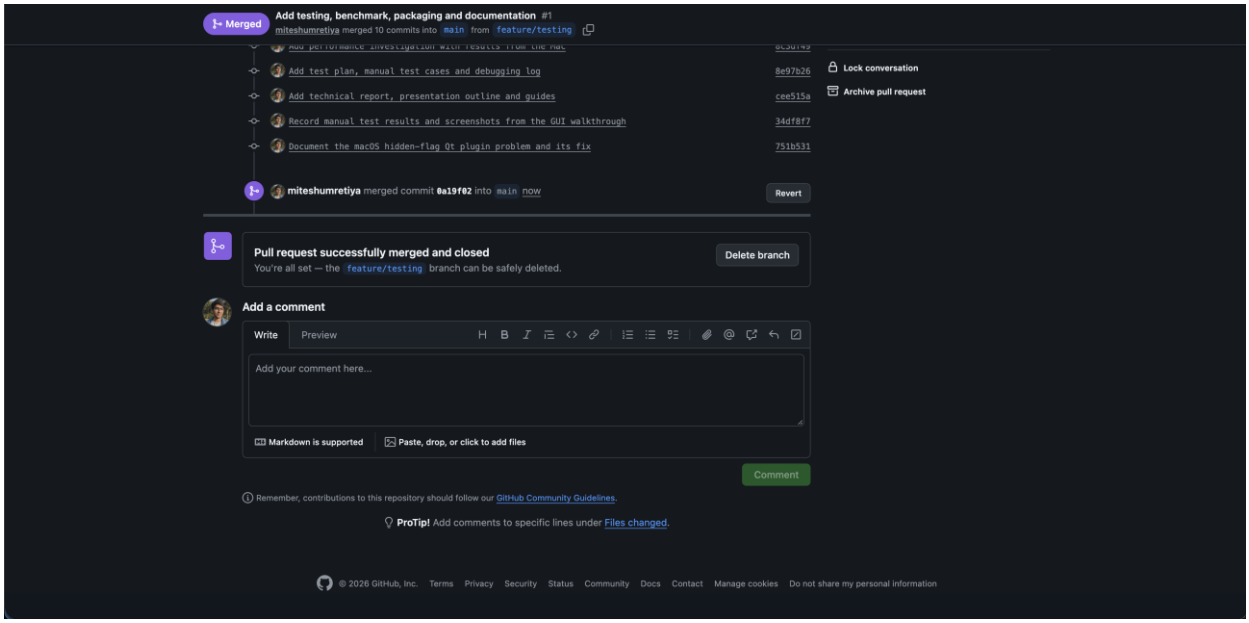



*Figure 5: Pull request #1 merged*

`git log --oneline --graph --all` shows the three branches splitting off and merging back in, including the conflict merge and the pull request merge:

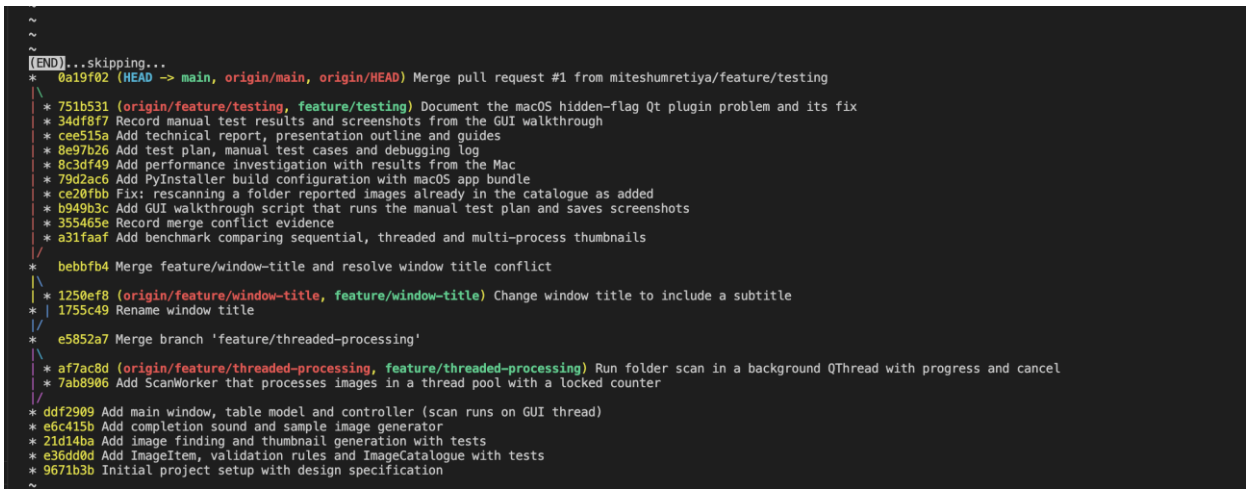



*Figure 6: Git history graph*

# 15.7 Evaluation

### Strengths

- Clear separation of concerns, which made testing easy: 68 automated tests run in under a second.
- The GUI stays responsive during long scans, with progress, cancel and a completion sound.
- Robust error handling: broken images, corrupt catalogue files and invalid input all give clear messages instead of crashes.
- Small and readable. Each file has one job.

### Limitations

- If you move or rename your photos, the catalogue still points to the old paths. There is no "relink" feature.
- Thumbnails are cached in the home folder and never cleaned up.
- The preview loads the full image on the GUI thread. With very large photos (e.g. 50 MP) selection can feel slightly slow.
- Only one folder can be scanned at a time.
- There are no automated GUI tests. GUI behaviour was checked manually.

### Technical problems I hit

- **Cancelling.** My first idea was to send a "cancel" signal to the worker. But the worker's thread is busy inside `run()`, so a queued signal would only be delivered after the scan had already finished. The fix was a `threading.Event` that the GUI thread sets directly and the pool threads check before each image.
- **Counting what was "added".** The status message counted every image the worker read, so rescanning a folder claimed images were added when they weren't (D5). The worker can't see the catalogue, by design, so the count had to move to the controller.
- **The GIL result surprised me.** I expected threads to be useless for CPU work in Python, but they were the fastest option. That's because Pillow releases the GIL. I had to change the benchmark (D4) to show the GIL's real effect on pure-Python code.
- **The app wouldn't start on my Mac at first.** It stopped with `Could not find the Qt platform plugin "cocoa"`, even though the plugin file was there. Using `QDir` in Python, I found that Qt could only see the file when hidden files were included. `ls -lO` showed

macOS had set the `hidden` flag on every file inside `.venv`, and Qt ignores hidden plugins. Running `chflags -R nohidden .venv` fixed it. The built `.app` was not affected because it lives in `dist/`. I added this to the README's troubleshooting section.

- **Packaging on macOS.** PyInstaller first produced a folder with a plain program file, not a normal Mac app. Adding a `BUNDLE` step to the spec file produced `ImageCatalogue.app`.

**Possible improvements**

- Load the preview in the background too, or use a medium-sized cached preview.
- Add rows to the table in batches (e.g. every 20 images) to reduce GUI work. This also reduces the sequential part in Amdahl's Law terms.
- Store the catalogue in SQLite instead of JSON for large collections.
- Add pytest-qt tests for the main window.
- Read EXIF data (date taken, camera) and allow sorting by date.

**Alternative approaches**

- `QThreadPool` + `QRunnable` instead of `QThread` + `ThreadPoolExecutor`. It is more "Qt", but QRunnable can't emit signals directly, so it needs an extra helper QObject.
- `ProcessPoolExecutor` for true parallelism. It is only worth it for pure-Python CPU work, and it adds pickling overhead and packaging complexity.
- `asyncio` (e.g. with qasync). It is good for I/O-bound work like downloads but doesn't help CPU-bound image decoding.

# References


- Riverbank Computing (2025) *PyQt6 Reference Guide*. Available at: https://www.riverbankcomputing.com/static/Docs/PyQt6/
- The Qt Company (2025) *Threads and QObjects*. Available at: https://doc.qt.io/qt-6/threads-qobject.html
- The Qt Company (2025) *Model/View Programming*. Available at: https://doc.qt.io/qt-6/model-view-programming.html
- Python Software Foundation (2025) *concurrent.futures: Launching parallel tasks*. Available at: https://docs.python.org/3/library/concurrent.futures.html
- Python Software Foundation (2025) *threading: Thread-based parallelism*. Available at: https://docs.python.org/3/library/threading.html
- Pillow contributors (2025) *Pillow documentation*. Available at: https://pillow.readthedocs.io/
- PyInstaller Development Team (2025) *PyInstaller Manual*. Available at: https://pyinstaller.org/en/stable/
- Amdahl, G. M. (1967) 'Validity of the single processor approach to achieving large scale computing capabilities', *AFIPS Conference Proceedings*, 30, pp. 483-485.
- Microsoft (2024) *Introduction to PLINQ*. Available at: https://learn.microsoft.com/en-us/dotnet/standard/parallel-programming/introduction-to-plinq

All online sources accessed 30 October 2026.